

Kapitel 11

Felder

11.1 Rückblick

Wir haben uns verschiedentlich bereits mit *Feldern* beschäftigt. Was wir dabei gelernt haben, ist im Folgenden noch einmal kurz zusammengefasst:

- Felder können durch

```
typ name[anzahl];
```

vereinbart werden. Der Wert *anzahl* muss ein konstanter Ausdruck mit ganzzahligem, positivem Wert sein.

- Die *Komponenten* (*Elemente*) eines Feldes sind Variablen mit dem Typ des Feldes. Sie können mit

```
name[index]
```

angesprochen werden. Da die Numerierung bei 0 beginnt, ist der größte zulässige *Index* um 1 kleiner als die Anzahl der Komponenten. Die Indizes selbst können beliebige Ausdrücke mit einem ganzzahligen Typ sein.

- Der Programmierer ist selbst dafür verantwortlich, dass er nur zulässige Indizes verwendet.
- Vektoren mit dem Typ **char** werden als Strings bezeichnet, wenn sie in einer Komponente ein Null-Zeichen enthalten.
- Feldnamen bezeichnen selbst keine Variablen, sondern sind Zeigerkonstanten. Das hat Auswirkungen auf die Übergabe von Feldern an Funktionen.

11.2 Vereinbarung von Feldern

Ähnlich wie eindimensionale Felder (*Vektoren*) können auch Felder mit mehreren Dimensionen vereinbart werden: Dem Namen folgen dann, jeweils in eckigen Klammern eingeschlossen, die Längenangaben für die verschiedenen Dimensionen. So ergibt

```
int m[2][3];
```

ein zweidimensionales Feld (*Matrix*). In gleicher Weise hat man mehrere Indizes anzugeben, wenn man auf ein einzelnes Element zugreifen will; im Beispiel hat man etwa die 6 Elemente

`m[0][0]`, `m[0][1]`, `m[0][2]`, `m[1][0]`, `m[1][1]`, `m[1][2]`.

Sowohl die Vereinbarung eines mehrdimensionalen Feldes als auch die Zugriffe auf seine Komponenten kann man auch in anderer Weise interpretieren: Eine Matrix zum Beispiel kann man auch als Vektor betrachten, dessen Komponenten ihrerseits Vektoren sind. Bei einem dreidimensionalen Feld hat man sogar drei Möglichkeiten der Interpretation (Vektor von Matrizen, Matrix von Vektoren, Vektor von Vektoren von Vektoren). Entsprechende Zugriffe sind erlaubt: `m[0]` und `m[1]` bezeichnen so die beiden Vektoren der Länge 3, aus denen `m` besteht. `m[1][0]` bezeichnet in diesem Sinne die erste Komponente im zweiten Vektor von `m`.

11.3 Anordnung von Feldern im Speicher

Für den C-Programmierer ist es in manchen Situationen unumgänglich zu wissen, wie Felder im Speicher angeordnet sind. Grundsätzlich wird für jedes Feld ein zusammenhängender Speicherbereich verwendet.

Bei Vektoren sind die Komponenten mit aufsteigenden Indizes hintereinander angeordnet: Den Index einer Komponente kann man als ihren Abstand vom ersten Element des Vektors interpretieren. Für den Vektor

```
int v[6];
```

ist die Anordnung der Komponenten in Abbildung 11.1 angegeben.

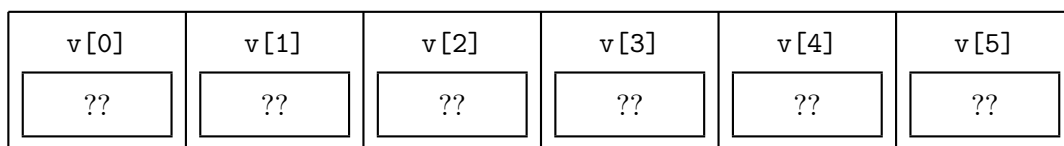


Abbildung 11.1: Speicheranordnung eines Vektors mit 6 Komponenten

Das ist übrigens auch der Grund, warum bei C Felder ab 0 indiziert werden. Bei der Realisierung speziell mathematischer Formeln kann das verwirren, weil in ihnen die Indizierung in der Regel *nicht* bei Null beginnt.

Die Beschreibung einer Matrix als Vektor von Vektoren sollte schon nahelegen, wie Matrizen angeordnet werden: Am Anfang steht der erste Zeilenvektor, dann der zweite Zeilenvektor, usw. Die Elemente der einzelnen Zeilenvektoren folgen jeweils unmittelbar hintereinander. Für die Matrix

```
int m[2][3];
```

ist dies in Abbildung 11.2 wiedergegeben.

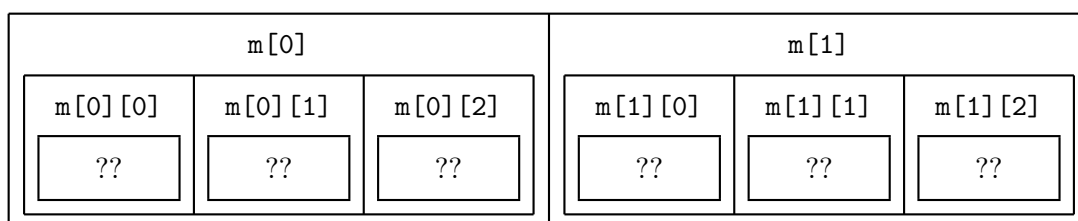


Abbildung 11.2: Speicheranordnung einer (2×3) -Matrix

Für den Zugriff auf eine Komponente einer Matrix muss der Abstand vom ersten Element jetzt wirklich „ausgerechnet“ werden. Haben wir etwa den Zugriff `m[i][j]`, so ist der Abstand gerade $i \cdot 3 + j$, oder allgemeiner: Zeilenindex mal Zeilenlänge plus Spaltenindex. Und genau diesen Ausdruck setzt der Compiler bei jedem Zugriff auf eine Matrixkomponente ein. Die Zuordnung $(i, j) \mapsto i \cdot 3 + j$ ist in dieser Situation die *Speicherabbildungsfunktion*. Ihre Gestalt hängt offenbar von der Dimensionierung der Matrix ab. Ähnliche Formeln kann man sich übrigens auch für Felder mit höherer Dimension überlegen. Dass jede Feldkomponente möglicherweise mehrere Bytes belegt und deshalb der Abstand noch einmal mit dieser Anzahl multipliziert werden muss, braucht uns nicht zu interessieren.

Damit ist jetzt auch schon angedeutet, was passiert, wenn man mit unzulässigen Indizes auf ein Feld zugreift: Der Abstand wird ausgerechnet und zum Zugriff verwendet. So bewirkt `v[-1]` gerade den Zugriff auf den Speicherbereich direkt *vor* dem Beginn des Vektors. Versuchte Zugriffe auf Feldkomponenten jenseits der Feldgrenzen nennt man *Indexüberschreitung*.

Ob und ggf. was für Folgen eine Indexüberschreitung hat, hängt von der Situation ab. Manche bleiben, von eventuell verfälschten Resultaten abgesehen, ohne Wirkung. So bewirkt zum Beispiel

```
m[0][3] = m[1][0];
```

auf *jeden Fall nichts*! Das Indexpaar `[0][3]` ist zwar nicht zulässig – da es aber denselben Abstand vom Anfang der Matrix liefert wie das Indexpaar `[1][0]`, nämlich 3, wird nur ein `int`-Wert mit sich selbst überschrieben.

Nicht jeder Verstoß endet so ohne Folgen. Da ist es fast schon angenehmer, wenn die Abstände so falsch sind, dass das Programm wegen Zugriffs auf einen „verbotenen“ Speicherbereich „gekillt“ wird, als wenn es mit verfälschten Daten weiterrechnet. An der Stelle, auf die zum Beispiel durch `v[-1]` zugegriffen wird, steht ja in der Regel eine andere Variable des Programms, die durch

```
v[-1] = 0;
```

verfälscht würde.

Wird ein Programm mit einer Meldung wie „segmentation fault“, „bus error“ bzw. „memory fault“ abgebrochen, sollte man das Programm zunächst auf Indexüberschreitungen hin untersuchen.

11.4 Felder als Parameter

Erinnerung: Die *Länge eines Strings*, der durch ein `char`-Feld realisiert ist, ist etwas anderes als die *Länge dieses Feldes*. Die Länge des Strings ist die Anzahl der Bytes vor dem Nullbyte im Feld. Die Länge des `char`-Feldes in dem die Zeichen des Strings gespeichert sind ist die Anzahl der Bytes, die das *komplette* Feld belegt.

Was eine Funktion dagegen nicht bestimmen kann, ist die Länge eines übergebenen Feldes. Wenn eine Funktion die Länge eines als Argument übergebenen Feldes benötigt, so muss ihr diese mit Hilfe eines separaten Parameters mitgeteilt werden. Selbst berechnen kann sie die Länge nämlich nicht, denn im Speicher folgt einfach Byte auf Byte – eine besondere Grenze, an der die Funktion erkennen könnte, dass das übergebene Feld „zu Ende“ ist, gibt es nicht.

Wegen des Unterschieds zwischen Länge eines Strings und Länge des Feldes durch den ein String realisiert ist, bilden Strings nur scheinbar eine Ausnahme. Als Beispiel betrachten wir eine Funktion, die die Summe der Komponenten eines Vektors berechnet:

```
float vektorsumme(const float v[], int laenge) {
    float s = 0;
    while (--laenge >= 0)
        s += v[laenge];
    return s;
}
```

Der Vorteil dieses Vorgehens ist, dass die Funktion für Vektoren beliebiger Länge eingesetzt werden kann: Der Funktion wird mitgeteilt, wo der Vektor beginnt (`v[]`). Sie unterstellt, dass weitere `laenge - 1` Komponenten folgen. Dass das so ist, ist wieder ausschließlich in die Verantwortung des Programmierers gestellt. Zulässig wäre zum Beispiel

```
float v1[7], v2[50], m[5][20], summe;
...
summe = vektorsumme(v1, 7) + vektorsumme(v2, 50);
summe = vektorsumme(m[3], 20);
```

Nicht zulässig wäre dagegen, bei gleichen Definitionen, wie bereits angesprochen

```
summe = vektorsumme(v1, 10);
```

da die Funktion auf Speicher zugreifen würde, der nicht zu `v1` gehört. Andererseits dürfen wir ohne weiteres

```
summe = vektorsumme(v2, 10);
summe = vektorsumme(&v2[10], 10);
```

schreiben – es steht uns ja frei, einen Teil der Komponenten von `v2` zu ignorieren, hier im ersten Fall die letzten 40 und im zweiten Fall die ersten 10 und die letzten 30. Und schließlich ist auch

```
summe = vektorsumme(m[0], 100);
```

zulässig. Das mag auf den ersten Blick verwunderlich erscheinen – spätestens auf den zweiten Blick wird es klar: Die Matrix `m` besteht aus insgesamt 100 Komponenten, die im Speicher unmittelbar aufeinanderfolgen. Was sollte uns also daran hindern, diesen Speicherbereich bei Bedarf auch als Vektor zu betrachten? Nur `m` als Parameter anzugeben reicht jedoch nicht aus. Ohne Zusatz bezeichnet `m` einen Vektor von Zeilenvektoren, nicht einen Vektor von `float`-Werten. `m[0]` bezeichnet den ersten Zeilenvektor, die weiteren Zeilenvektoren folgen jedoch unmittelbar im Speicher.

Schwieriger wird es, wenn ein mehrdimensionales Feld auch in der Funktion als solches betrachtet werden muss – und wenn obendrein die Größe des Feldes in den einzelnen Dimensionen von Aufruf zu Aufruf verschieden können sein soll. In Deklaration und Definition einer Funktion dürfen wir Felder zwar auch „echt“ dimensionieren, d.h. in die Klammern etwas eintragen – nur müssen das Konstanten oder konstante Ausdrücke sein, deren Wert dann natürlich von Aufruf zu Aufruf nicht verändert werden kann. Andererseits dürfen wir in einer Funktionsvereinbarung nur für die erste Dimension die Längenangabe weglassen. Der Prototyp

```
float f(float m[][3], int zeilen);
```

ist also zwar zulässig, erlaubt den Einsatz der Funktion `f` aber nur für Matrizen mit 3 Spalten – und ist deshalb für die Praxis unzureichend. Der Prototyp

```
float f(float m[][], int zeilen, int spalten);
```

ist *nicht* erlaubt. Der Grund sollte klar sein: Ohne die Längenangabe der Zeilen hat der Compiler keine Möglichkeit, die Speicherabbildungsfunktion zu berechnen.

Die Abhilfe: Man übergibt die Matrix formal als Vektor und gibt diesem Vektor in der Funktion wieder Matrixstruktur, indem man die Speicherabbildungsfunktion selbst berechnet. Das ist zwar nicht schön, in C aber nicht zu vermeiden.

Ein Beispiel ist das Matrizenprodukt $C = A \cdot B$ für Matrizen $A = (a_{ij})$, $B = (b_{jk})$ und $C = (c_{ik})$ mit $i = 1, \dots, \ell$, $j = 1, \dots, m$ und $k = 1, \dots, n$:

$$c_{ik} = \sum_{j=1}^m (a_{ij} \cdot b_{jk})$$

Der Prototyp einer entsprechenden Funktion kann

```
void matprod(float c[], const float a[], const float b[],
             int l, int m, int n);
```

sein. Bevor wir die Funktion definieren, deklarieren wir zunächst einen Makro, der zu einem Indexpaar den Abstand vom Anfang des Vektors berechnet. Dieser Makro benötigt zusätzlich zu den beiden Indizes offensichtlich auch die Länge der Zeilen der Matrix als Parameter. In diesen Makro können wir dann auch gleich noch die Transformation der logischen Indizes $1, \dots, x$ in die C-Indizes $0, \dots, x - 1$ hineinstecken:

```
#define lin(zeile, spalte, laenge) \
    (((zeile)-1)*(laenge)+(spalte)-1)
```

Jetzt können wir die Funktion definieren – in einer Schreibweise, die der Formel zumindest einigermaßen nahekommt:

```
void matprod (float c[], const float a[], const float b[],
              int l, int m, int n) {
    int i, j, k;
    for (i = 1; i <= l; i++)
        for (k = 1; k <= n; k++) {
            c[lin(i,k,n)] = 0;
            for (j = 1; j <= m; j++)
                c[lin(i,k,n)] += a[lin(i,j,m)] * b[lin(j,k,n)];
        }
}
```

Diese Formulierung entspricht in etwa der Schreibweise der Formel, dafür aber nicht so sehr der Schreibweise von C. Dieses Beispiel wird deshalb im nächsten Abschnitt noch einmal aufgegriffen.

Ein Aufruf der Funktion könnte so aussehen:

```
#define DIM_L ...
#define DIM_M ...
#define DIM_N ...
/* ... */
float A[DIM_L][DIM_M], B[DIM_M][DIM_N],
      C[DIM_L][DIM_N];
/* ... Initialisierung der Matrizen ... */

matprod (C[0], A[0], B[0], DIM_L, DIM_M, DIM_N);
```

Im übrigen lässt sich auch diese Routine noch nicht wirklich universell für die Multiplikation von zwei `float`-Matrizen einsetzen: Ein Programm, das Matrizen verarbeitet, soll in der Regel bei jedem Aufruf Matrizen unterschiedlicher Größe verarbeiten können. Um das zu realisieren, gibt es in C zwei Möglichkeiten:

- Man arbeitet mit *dynamischem Speicher*. Das bedeutet, man bestimmt zunächst die Größe der zu bearbeitenden Matrizen und beschafft sich dann vom Betriebssystem den entsprechenden Speicher. Methoden dazu werden im Kapitel 13 vorgestellt. Der Einsatz unserer Funktion `matprod` wäre dann möglich.
- Man arbeitet mit „Maximalgrößen“ für die Matrizen, d.h. man überlegt sich eine obere Schranke für die Größe der vorkommenden Matrizen und definiert alle Matrizen mit dieser Größe. Das wird in mathematischen Programmen oft gemacht, hat aber auch seine Nachteile:
 - Sollte der Benutzer des Programms größere Matrizen verarbeiten wollen, muss die Verarbeitung abgelehnt werden.
 - Funktionen wie `matprod` benötigen zusätzliche Parameter: Für die Schleifen wird die „logische“ (genutzte) Größe der Matrizen benötigt; für die Berechnung der Speicherabbildungsfunktion wird dagegen die „physikalische“ (vereinbarte) Größe der Matrizen benötigt.

Leider fehlt die Zeit, auf Einzelheiten einzugehen. Vielleicht formulieren Sie selbst die Funktion `matprod` einmal so um, dass sie auch dann korrekt arbeitet, wenn die Matrizen nur einen Teil des für sie bereitgestellten Speichers belegen.

11.5 Initialisierung von Feldern

Felder können initialisiert werden. Für Strings haben wir diese Möglichkeit auch bereits genutzt. Allgemein sieht die Anfangswertzuweisung für Felder aber anders aus: Die Werte der Komponenten werden einzeln aufgeführt, voneinander jeweils durch ein Komma getrennt und insgesamt in geschweifte Klammern eingeschlossen:

```
int v[6] = {6, 5, 4, 17, 18, 19};
```

Gibt man zu wenig Werte an, so werden die übrigen auf Null gesetzt; gibt man zu viele Werte an, wird der Compiler einen Fehler anzeigen. Wenn man für jede Komponente einen Wert angibt, kann man den Compiler auch abzählen lassen und die Feldlänge ganz weglassen: Die Definition

```
int v[] = {6, 5, 4, 17, 18, 19};
```

ist also zulässig und ergibt einen Vektor mit 6 Komponenten.

Bei der Initialisierung mehrdimensionaler Felder muss man sich wieder klar machen, dass sie formal Vektoren von Vektoren sind. Entsprechend kann man dann zum Beispiel

```
int m[2][3] = {{ 6, 5, 4},
               {17, 18, 19}};
```

schreiben.

Wenn man Anfangswerte angibt, darf man bei mehrdimensionalen Feldern die Länge in der ersten Dimension weglassen; der Compiler bestimmt sie dann durch Abzählen. Das ist nicht nur für Schreibfaule interessant, sondern erspart einem einerseits das Abzählen der Daten und verhindert andererseits ein Verzählen dabei.

Auch mehrdimensionale Felder lassen sich teilweise initialisieren. Alle nicht angegebenen Komponenten erhalten dann den Wert 0. Dabei wird die durch geschweiften Klammern angegebene Struktur eingehalten.

```
float v[12] = { 0 };    /* alle Komponenten = 0 */

int m[4][2][3] = {{{ 1, 2, 3 }, { 4, 5, 6 }},
                  {{ 7, 8 }},
                  {{ 9 }, { 10 }}}};
/* entspricht = {{{ 1, 2, 3 }, { 4, 5, 6 }},
                  {{ 7, 8, 0 }, { 0, 0, 0 }},
                  {{ 9, 0, 0 }, { 10, 0, 0 }},
                  {{ 0, 0, 0 }, { 0, 0, 0 }}} */
```

Im Beispiel hätte man wiederum die erste Dimension des Feldes `m` weglassen können. Der Compiler hätte für sie dann 3 bestimmt, da für drei Zeilen Anfangswerte angegeben wurden.

Für Strings haben wir bereits eine Kurzform der Initialisierung kennengelernt: Wenn eine Stringkonstante angegeben ist, zerlegt der Compiler diese in einzelne Zeichen und sorgt selber für das abschließende Null-Zeichen. Die beiden Definitionen

```
char str1[6] = "hallo",
      str2[6] = {'h', 'a', 'l', 'l', 'o', '\0'};
```

sind also äquivalent. Und da wir auch hier wieder den Compiler abzählen lassen können, sind die beiden Definitionen

```
char str3[] = "hallo",
      str4[] = {'h', 'a', 'l', 'l', 'o', '\0'};
```

ebenfalls äquivalent.

Alle Konstanten, die in den Initialisierungen angegeben wurden, dürften übrigens auch konstante Ausdrücke sein, bei automatischen Feldern sogar beliebige Ausdrücke.

Ein Manko ist sicher, dass man alle Anfangswerte explizit angeben muss. Eine Abkürzung wie „trage 10mal den Wert 7 ein“ gibt es nicht! Auch kann man nicht nur für einzelne Feldkomponenten Anfangswerte angeben.

